

Problem 1:

```
import math
import matplotlib.pyplot as plt

def build_domain(a, b, c, disc):
    if disc >= 0:
        sqrt_disc = math.sqrt(disc)
        x1 = (-b - sqrt_disc) / (2 * a)
        x2 = (-b + sqrt_disc) / (2 * a)
        left_root = min(x1, x2)
        right_root = max(x1, x2)
        width = right_root - left_root

        if width == 0:
            margin = 2.0
        else:
            margin = width

        xmin = left_root - margin
        xmax = right_root + margin
    else:
        xopt = -b / (2 * a)
        xmin = xopt - 5.0
        xmax = xopt + 5.0

    return xmin, xmax

def plot_quadratic(a, b, c, xmin, xmax):
    n = 150
    step = (xmax - xmin) / (n - 1)

    xs = []
    ys = []

    for i in range(n):
        x = xmin + i * step
        y = a * x ** 2 + b * x + c
        xs.append(x)
        ys.append(y)

    plt.figure()
    plt.plot(xs, ys, marker='o', markersize=2)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title(f'y = {a}x^2 + {b}x + {c}')
```

```

plt.grid(True)
plt.show()

while True: a_text = input('Enter a: ')if a_text == "": break

b_text = input('Enter b: ')
c_text = input('Enter c: ')

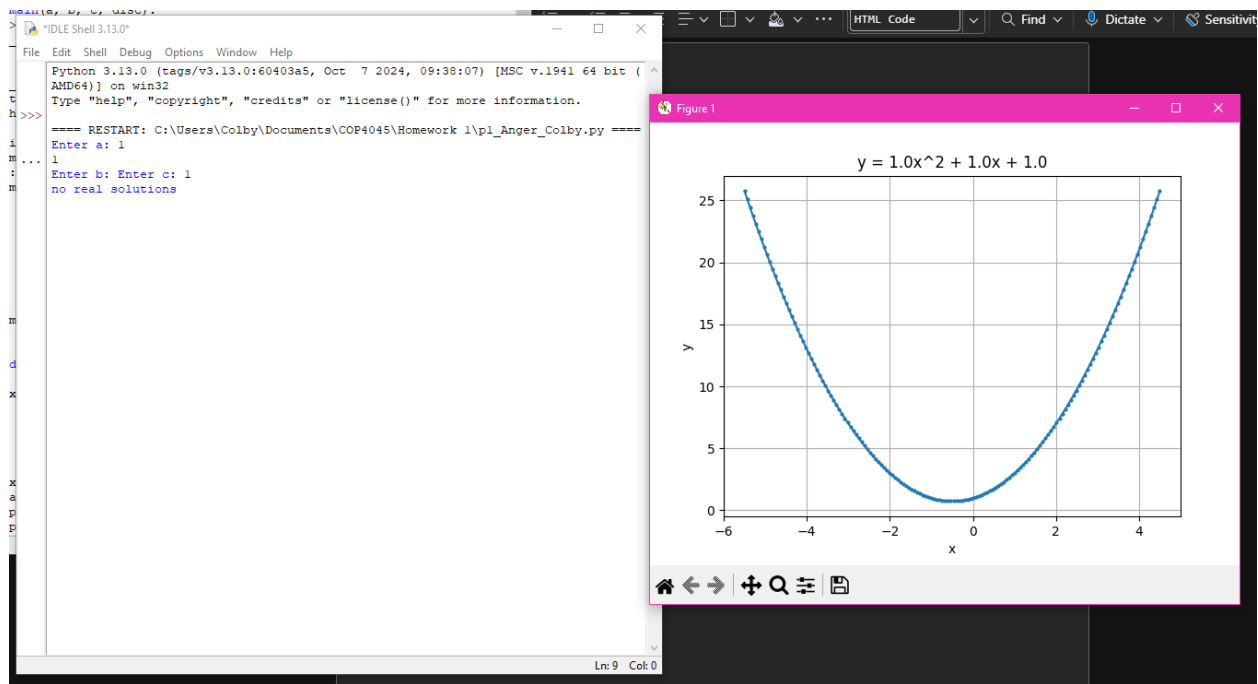
a = float(a_text)
b = float(b_text)
c = float(c_text)

disc = b ** 2 - 4 * a * c

if disc < 0:
    print('no real solutions')
elif disc == 0:
    x1 = -b / (2 * a)
    print(f'one solution: {x1:.5f}')
else:
    sqrt_disc = math.sqrt(disc)
    x1 = (-b - sqrt_disc) / (2 * a)
    x2 = (-b + sqrt_disc) / (2 * a)
    print(f'two solutions: x1={x1:.5f} x2={x2:.5f}')

xmin, xmax = build_domain(a, b, c, disc)
plot_quadratic(a, b, c, xmin, xmax)

```



Problem 2:

```
def find_Pythagorean(n): triples = []

for a in range(1, n + 1):
    for b in range(1, n + 1):
        for c in range(1, n + 1):
            if a * a + b * b == c * c:
                triples.append((a, b, c))

return triples

n = int(input('Enter a positive integer n: ')) triples = find_Pythagorean(n)

for triple in triples: print(triple)
```

Problem 3:

```
def find_dup_str(s, n):

    if n <= 0 or n > len(s):

        return "
```

```
for i in range(len(s) - n + 1):  
    part1 = s[i:i + n]  
  
    for j in range(i + n, len(s) - n + 1):  
        part2 = s[j:j + n]  
  
        if part1 == part2:  
            return part1  
  
return "
```

```
def find_max_dup(s):  
    for n in range(len(s) // 2, 0, -1):  
        dup = find_dup_str(s, n)  
        if dup != "":  
            return dup  
  
    return "
```

```
s = input('Enter a string: ')  
n = int(input('Enter substring length: '))  
print(find_dup_str(s, n))
```

```
s = input('Enter a string for maximum duplicated substring: ')
print(find_max_dup(s))
```

Problem 4:

```
import math
import matplotlib.pyplot as plt
```

```
def plot_function(fun_str, domain, ns):
```

```
    xmin, xmax = domain
```

```
    step = (xmax - xmin) / (ns - 1)
```

```
    xs = []
```

```
    ys = []
```

```
    for i in range(ns):
```

```
        x = xmin + i * step
```

```
        y = eval(fun_str)
```

```
        xs.append(x)
```

```
        ys.append(y)
```

```
    print('{:>10s} {:>10s}'.format('x', 'y'))
```

```
    print('-' * 21)
```

```
    for i in range(ns):
```

```
print('{:>10.4f} {:>+10.4f}'.format(xs[i], ys[i]))
```

```
plt.figure()
```

```
plt.plot(xs, ys, marker='o', markersize=3)
```

```
plt.xlabel('x')
```

```
plt.ylabel('y')
```

```
plt.title(fun_str)
```

```
plt.grid(True)
```

```
plt.show()
```

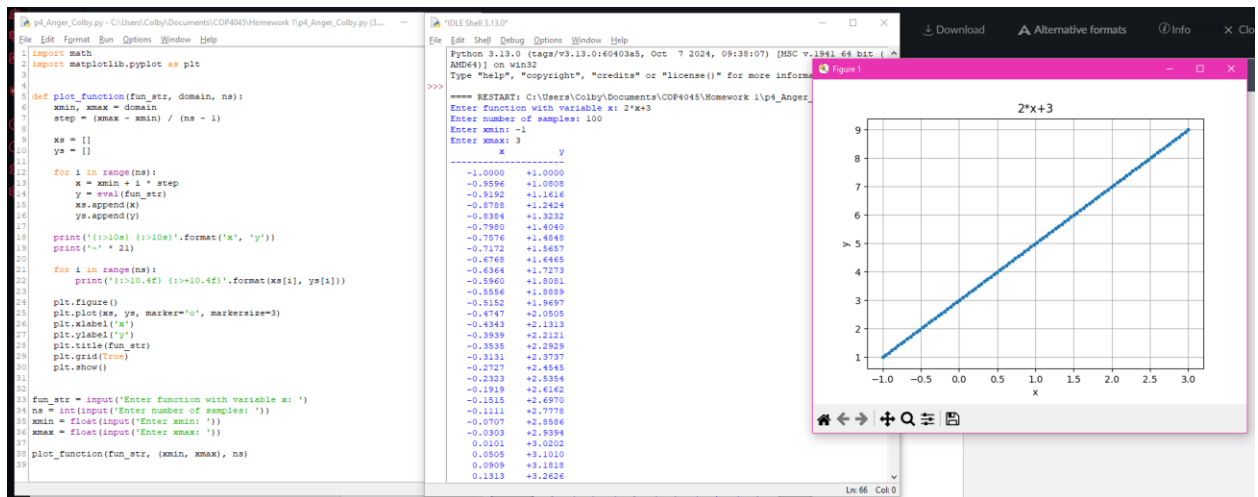
```
fun_str = input('Enter function with variable x: ')
```

```
ns = int(input('Enter number of samples: '))
```

```
xmin = float(input('Enter xmin: '))
```

```
xmax = float(input('Enter xmax: '))
```

```
plot_function(fun_str, (xmin, xmax), ns)
```



Problem 5:

Problem 5. Interactive Caesar Cipher with Coding Agent Help

For this problem you must use ChatGPT Codex or Copilot to solve the problem.

This problem evaluates your use of a coding agent.

Write a program in a file `p5_.....py` that interacts with the user via the terminal to encrypt/decrypt text and analyze letter frequencies.

- a) Write a function `caesar_cipher(text, shift)` that shifts letters in a string by a given integer using string indexing and loops, preserving spaces and casing, and that returns the shifted (encrypted) string.
- b) Write a function `caesar_decipher(cyphertext, shift)` that decrypts a Caesar-encrypted string and that return the original string clear string.
- c) Write a function `letter_frequency(text)` that counts how many times each letter of the alphabet appears in the text, ignoring case and non-alphabetic characters.
- d) Create a `main()` function with a terminal menu allowing the user to type a message, specify a shift value, and view both the ciphered text, the letter frequency breakdown, and the deciphered text.
- e) Write a separate test file `p5_<your name>_test.py` using Python's `unittest` framework to verify your cipher and frequency functions.



I have this school assignment that I would like to complete with your help.

I have this school assignment that I would like to complete with your help.

Absolutely, Colby — I can help you build this entire assignment step-by-step.

Below is everything you need: the **main program**, the **cipher/frequency functions**, and the **unittest file**.

You can copy these into your `p5_<your name>.py` and `p5_<your name>_test.py` files.

✓ Your main program: `p5_colby.py`

Python ^

Copy

```
# p5_colby.py

import string

def caesar_cipher(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            alphabet = string.ascii_uppercase if char.isupper() else string.ascii_lowercase
            idx = alphabet.index(char)
            shifted_idx = (idx + shift) % 26
            result += alphabet[shifted_idx]
        else:
            result += char
    return result

def caesar_decipher(ciphertext, shift):
```

Message Copilot



Smart v



p5AngerColby.py

```
import string
```

```
def caesar_cipher(text, shift): result = ""for char in text: if char.isalpha(): alphabet =
string.ascii_uppercase if char.isupper() else string.ascii_lowercase idx =
alphabet.index(char) shifted_idx = (idx + shift) % 26 result += alphabet[shifted_idx]
else: result += charreturn result
```

```
def caesar_decipher(ciphertext, shift): return caesar_cipher(ciphertext, -shift)
```

```
def letter_frequency(text): freq = {letter: 0 for letter in string.ascii_lowercase}for char in text.lower(): if char in freq: freq[char] += 1return freq
```

```
def main(): print("=== Interactive Caesar Cipher Tool ===")
```

```
while True:
```

```
    print("\nMenu:")
```

```
    print("1. Encrypt text")
```

```
    print("2. Decrypt text")
```

```
    print("3. Analyze letter frequency")
```

```
    print("4. Quit")
```

```
choice = input("Choose an option (1-4): ")
```

```
if choice == "1":
```

```
    text = input("Enter text to encrypt: ")
```

```
    shift = int(input("Enter shift value: "))
```

```
    encrypted = caesar_cipher(text, shift)
```

```
    print("\nEncrypted text:", encrypted)
```

```
    print("\nLetter frequency:")
```

```
    for letter, count in letter_frequency(encrypted).items():
```

```
        print(f"{letter}: {count}")
```

```
    print("\nDeciphered back:", caesar_decipher(encrypted, shift))
```

```
elif choice == "2":
```

```
    text = input("Enter text to decrypt: ")
```

```
    shift = int(input("Enter shift value: "))
```

```
    decrypted = caesar_decipher(text, shift)
```

```
    print("\nDecrypted text:", decrypted)
```

```
elif choice == "3":
```

```
    text = input("Enter text to analyze: ")
```

```
    freq = letter_frequency(text)
```

```
    print("\nLetter frequency:")
```

```
    for letter, count in freq.items():
```

```
        print(f"{letter}: {count}")
```

```
elif choice == "4":
```

```
    print("Goodbye!")
```

```
break
```

```
else:
```

```
    print("Invalid choice. Try again.")
```

```
if name == "main": main()
```

P5AngerColby_Test.py

```
import unittest from p5_Anger_Colby import caesar_cipher, caesar_decipher,  
letter_frequency
```

```
class TestCaesarCipher(unittest.TestCase):
```

```
    def test_cipher_basic(self):
```

```
        self.assertEqual(caesar_cipher("abc", 1), "bcd")
```

```
        self.assertEqual(caesar_cipher("xyz", 3), "abc")
```

```
    def test_cipher_preserves_case(self):
```

```
        self.assertEqual(caesar_cipher("AbC", 2), "CdE")
```

```
    def test_decipher(self):
```

```
        encrypted = caesar_cipher("Hello World", 5)
```

```
        decrypted = caesar_decipher(encrypted, 5)
```

```
        self.assertEqual(decrypted, "Hello World")
```

```
    def test_letter_frequency(self):
```

```
        freq = letter_frequency("AaBbCc!!")
```

```
        self.assertEqual(freq["a"], 2)
```

```
        self.assertEqual(freq["b"], 2)
```

```
        self.assertEqual(freq["c"], 2)
```

```
        self.assertEqual(freq["d"], 0)
```

```
if name == "main": unittest.main()
```